Declaration of Generative AI Usage

SABD 2025/26 — Project 1

Daniel Garoz — Erasmus Exchange, Università degli Studi di Roma "Tor Vergata"

Scope of this declaration

This document accompanies the Project 1 report and discloses the use of generative AI tools during the development of the project, following the updated guidelines communicated by Prof. V. Cardellini on **June 2, 2026**.

1. Tools used

The only generative AI tool used during this project is:

- **Anthropic Claude**, accessed through the standard chat interface. Used as an interactive assistant throughout the development of the project.

No other generative AI tool (Copilot, Cursor, GPT-based coders, auto-completion plugins, etc.) was used at any point.

2. Activities for which the AI was used

The AI was used as a productivity and learning aid for the following activities:

- **Project design and brainstorming**: discussion of the overall architecture (HDFS + Spark cluster on Docker Compose), identification of the relevant columns, choice of design patterns (e.g. schema-on-load, cache policy, shuffle partitions), and exploration of trade-offs.
- **Code generation**: initial drafts of the Python modules in `src/` (`utils.py`, `preprocessing.py`, `query1.py`, `query2.py`, `plot_q1.py`, `plot_q2.py`, `benchmark.py`) and of the Docker layer (`docker-compose.yml`, `hadoop.env`, `ingest_to_hdfs.sh`, `run_on_hdfs.sh`, `pull_results.sh`).
- **Debugging and troubleshooting**: WSL2 DNS issues, PySpark installation, Docker Compose plugin installation, LaTeX compilation errors, schema mismatches when stray BTS lookup files were present, identification of duplicated counts.
- **Report writing**: initial structure and drafting of the IEEE-format report (Sections I–VII), and subsequent updates when the four-month dataset became available.
- **Slides drafting**: initial Beamer slide deck for the oral presentation.
- **Documentation**: drafting of `README.md` and docstrings.
- **Learning and oral defence preparation**: line-by-line walkthroughs of every script, conceptual discussions on Spark internals (lazy evaluation, narrow/wide transformations, cache, shuffle, DAG), HDFS architecture, Docker, and benchmark methodology; multiple mock oral defence sessions with question-and-answer drilling.

3. Parts of the project significantly generated or modified by AI

- All Python source files in `src/` were drafted by the AI from natural-language specifications I provided, then reviewed line by line.
- The Docker Compose stack and the supporting shell scripts were drafted by the AI, then tested and adjusted.
- The body of the report (LaTeX source) was drafted by the AI and then reviewed, restructured, and edited.
- The Beamer source of the slides was drafted by the AI.
- Plot configurations (axis labels, colour schemes, layouts) in `plot_q1.py` and `plot_q2.py` are AI drafts.

The numerical results in the report (means, standard deviations, percentage overheads, dataset sizes, ranking of carriers) were **not generated by the AI**: they are the verbatim output of the benchmark and query runs that I executed locally and on the HDFS cluster on my own machine.

4. Review, modification and validation performed by the student

Although the AI provided the initial drafts of much of the technical content, every artefact was reviewed and validated by me before being included in the final deliverables. Specifically:

- I read every line of source code, understood its purpose, and modified styling, comments and structure where needed. I am able to explain and defend every design decision in the oral examination.
- I executed the pipeline end-to-end multiple times — both in the local PySpark environment and on the dockerized HDFS+Spark cluster — and **validated the outputs against the project specification** (number of result rows, shape of the CSV outputs, correctness of the cancellation rate computation, sensible ranking in Q2, presence of HDFS `_SUCCESS` markers, etc.).
- I diagnosed and resolved a number of real-world issues myself (incomplete dataset archive, presence of macOS metadata files, presence of unrelated BTS lookup files that inflated Q2 counts, missing Docker Compose plugin, DNS resolution problems in WSL2).
- I designed and executed the benchmark protocol (warm-up + 5 measured iterations, two configurations) and produced the numbers reported in Tables II and III of the report.
- I manually edited the report and the slides for tone, correctness, and consistency, and updated them when the dataset grew from three to four months.
- I prepared the repository structure (renaming to `Results/` and `Report/` as required by the specification, populating `requirements.txt`, managing the `.gitignore`, pushing to GitHub).

5. Preparation for the oral defence

I have spent dedicated study sessions on the project after the code was complete: per-file walkthroughs, deep dives on Spark internals (narrow vs. wide transformations, the role of `cache()`, the shuffle, the optimizer), HDFS architecture (NameNode/DataNode flow, replication, SPOF), Docker Compose (named volumes vs. bind mounts, networks, `depends_on` vs. `SERVICE_PRECONDITION`), and several rounds of mock oral examination.

I am prepared to discuss and justify every design decision, every line of code, and every number in the report, regardless of whether that content was first drafted by the AI.

Daniel Garoz — Erasmus Exchange Student
M.Sc. Computer Engineering
Università degli Studi di Roma "Tor Vergata"
A.A. 2025/26 — SABD Project 1
Submitted: 5 June 2026